

Guia de metodos de arrays en JavaScript

Una referencia practica para agregar, buscar, transformar, ordenar y validar datos con arrays.

Incluye tablas resumidas, ejemplos de uso y buenas practicas para elegir el metodo adecuado.

Autor: ChatGPT

Formato: guia rapida para estudiantes y desarrolladores junior.

1. Conceptos clave antes de usar metodos de arrays

Un array en JavaScript es una estructura ordenada de valores. Cada elemento tiene un indice que inicia en 0. Muchos metodos reciben una funcion callback, por ejemplo `elemento => condicion` o `(acumulador, elemento) => nuevoValor`.

Hay una diferencia importante entre metodos que modifican el array original y metodos que devuelven una copia o un nuevo valor. Cuando trabajes con datos en interfaces, estados o funciones puras, suele ser mas seguro preferir metodos que no mutan.

Mutan el array original	No mutan el array original
push, pop, shift, unshift, splice, sort, reverse, fill, copyWithin	map, filter, reduce, slice, concat, join, flat, flatMap, find, includes, some, every, toSorted, toReversed, toSpliced, with

Regla mental: si el metodo empieza por `to` en los metodos modernos, normalmente devuelve una copia: `toSorted()`, `toReversed()`, `toSpliced()`.

2. Lista de metodos por categoria

Agregar o quitar elementos

Metodo	Uso principal	Efecto	Ejemplo
push()	Agrega uno o mas elementos al final.	Modifica el original	<code>arr.push(4)</code>
pop()	Elimina y devuelve el ultimo elemento.	Modifica el original	<code>arr.pop()</code>
unshift()	Agrega uno o mas elementos al inicio.	Modifica el original	<code>arr.unshift(0)</code>
shift()	Elimina y devuelve el primer elemento.	Modifica el original	<code>arr.shift()</code>
splice()	Agrega, elimina o reemplaza elementos en una posicion.	Modifica el original	<code>arr.splice(1, 2, "x")</code>
slice()	Devuelve una copia parcial del array.	No modifica	<code>arr.slice(1, 3)</code>

Recorrer, transformar y acumular

Metodo	Uso principal	Efecto	Ejemplo
forEach()	Ejecuta una funcion por cada elemento.	No retorna array nuevo	<code>arr.forEach(x => console.log(x))</code>
map()	Transforma cada elemento y devuelve un nuevo array.	No modifica	<code>arr.map(x => x * 2)</code>
filter()	Devuelve los elementos que cumplen una condicion.	No modifica	<code>arr.filter(x => x > 10)</code>
reduce()	Acumula los elementos en un unico valor.	No modifica	<code>arr.reduce((a, x) => a + x, 0)</code>
reduceRight()	Acumula de derecha a izquierda.	No modifica	<code>arr.reduceRight((a, x) => a + x, "")</code>
flatMap()	Mapea cada elemento y aplana un nivel.	No modifica	<code>arr.flatMap(x => [x, x * 2])</code>

Buscar elementos

Metodo	Uso principal	Efecto	Ejemplo
find()	Devuelve el primer elemento que cumple una condicion.	No modifica	<code>arr.find(u => u.id === 2)</code>
findIndex()	Devuelve el indice del primer elemento que cumple.	No modifica	<code>arr.findIndex(x => x < 0)</code>
findLast()	Devuelve el ultimo elemento que cumple.	No modifica	<code>arr.findLast(x => x > 10)</code>
findLastIndex()	Devuelve el indice del ultimo elemento que cumple.	No modifica	<code>arr.findLastIndex(x => x > 10)</code>
includes()	Indica si el array contiene un valor.	No modifica	<code>arr.includes("js")</code>

Metodo	Uso principal	Efecto	Ejemplo
indexOf()	Devuelve el primer indice de un valor.	No modifica	<code>arr.indexOf("js")</code>
lastIndexOf()	Devuelve el ultimo indice de un valor.	No modifica	<code>arr.lastIndexOf("js")</code>

Ordenar y reordenar

Metodo	Uso principal	Efecto	Ejemplo
sort()	Ordena los elementos. Conviene pasar comparador para numeros.	Modifica el original	<code>arr.sort((a, b) => a - b)</code>
reverse()	Invierte el orden de los elementos.	Modifica el original	<code>arr.reverse()</code>
toSorted()	Devuelve una copia ordenada.	No modifica	<code>arr.toSorted((a, b) => a - b)</code>
toReversed()	Devuelve una copia invertida.	No modifica	<code>arr.toReversed()</code>

Combinar, convertir y aplanar

Metodo	Uso principal	Efecto	Ejemplo
concat()	Une arrays o valores y devuelve un nuevo array.	No modifica	<code>a.concat(b)</code>
join()	Une los elementos en un string.	No modifica	<code>arr.join(", ")</code>
flat()	Aplana arrays anidados.	No modifica	<code>arr.flat(2)</code>
Array.from()	Crea un array desde un iterable o array-like.	Estatico	<code>Array.from("hola")</code>
Array.of()	Crea un array con los argumentos dados.	Estatico	<code>Array.of(1, 2, 3)</code>

Validar condiciones

Metodo	Uso principal	Efecto	Ejemplo
some()	Verifica si al menos un elemento cumple.	No modifica	<code>arr.some(x => x > 18)</code>
every()	Verifica si todos los elementos cumplen.	No modifica	<code>arr.every(x => x > 0)</code>

Copiar o rellenar

Metodo	Uso principal	Efecto	Ejemplo
fill()	Rellena posiciones con un valor.	Modifica el original	<code>arr.fill(0)</code>
copyWithin()	Copia una parte del array dentro del mismo array.	Modifica el original	<code>arr.copyWithin(0, 2, 4)</code>

Metodo	Uso principal	Efecto	Ejemplo
with()	Devuelve una copia cambiando un elemento por indice.	No modifica	<code>arr.with(1, "nuevo")</code>
toSpliced()	Devuelve una copia aplicando una operacion tipo splice.	No modifica	<code>arr.toSpliced(1, 1, "x")</code>

Iteradores y utilidades

Metodo	Uso principal	Efecto	Ejemplo
keys()	Devuelve un iterador con los indices.	No modifica	<code>arr.keys()</code>
values()	Devuelve un iterador con los valores.	No modifica	<code>arr.values()</code>
entries()	Devuelve un iterador de pares [indice, valor].	No modifica	<code>arr.entries()</code>
Array.isArray()	Verifica si un valor es un array.	Estatico	<code>Array.isArray(arr)</code>

3. Ejemplos practicos

Estos ejemplos muestran casos comunes: transformar datos, filtrar resultados, sumar valores, buscar objetos y ordenar numeros.

map()

```
const precios = [10, 20, 30];
const conIva = precios.map(precio => precio * 1.19);
console.log(conIva); // [11.9, 23.8, 35.7]
```

filter()

```
const edades = [12, 18, 25, 15];
const mayores = edades.filter(edad => edad >= 18);
console.log(mayores); // [18, 25]
```

reduce()

```
const ventas = [100, 250, 80];
const total = ventas.reduce((acum, venta) => acum + venta, 0);
console.log(total); // 430
```

find()

```
const usuarios = [
  { id: 1, nombre: "Ana" },
  { id: 2, nombre: "Luis" }
];
const usuario = usuarios.find(u => u.id === 2);
console.log(usuario.nombre); // Luis
```

sort()

```
const numeros = [10, 2, 30, 4];
numeros.sort((a, b) => a - b);
console.log(numeros); // [2, 4, 10, 30]
```

toSorted()

```
const numeros = [3, 1, 2];
const ordenados = numeros.toSorted((a, b) => a - b);
console.log(ordenados); // [1, 2, 3]
console.log(numeros); // [3, 1, 2]
```

4. Como elegir el metodo correcto

Situacion	Metodo recomendado	Comentario
Necesitas transformar cada elemento	Usa map()	Devuelve un nuevo array con la misma cantidad de elementos.
Necesitas quedarte con algunos elementos	Usa filter()	Devuelve un nuevo array con los elementos que cumplen la condicion.
Necesitas un total, promedio, objeto o valor unico	Usa reduce()	Convierte el array en un solo resultado.
Necesitas encontrar un elemento especifico	Usa find()	Devuelve el primer elemento que cumple la condicion.
Necesitas saber si existe un valor simple	Usa includes()	Ideal para strings, numeros y booleanos.
Necesitas validar una regla en todos los elementos	Usa every()	Devuelve true solo si todos cumplen.
Necesitas validar si alguno cumple	Usa some()	Devuelve true si al menos uno cumple.
Necesitas ordenar sin tocar el original	Usa toSorted()	Mas seguro que sort() cuando debes conservar el array inicial.
Necesitas reemplazar por indice sin mutar	Usa with()	Alternativa moderna a cambiar <code>arr[indice]</code> directamente.

Buenas practicas

- Usa nombres claros en los callbacks: producto, usuario, precio, venta, etc.
- Para ordenar numeros, siempre usa comparador: `numeros.sort((a, b) => a - b)`.
- Evita mutar arrays cuando trabajes con estados de frameworks como React; prefiere `map`, `filter`, `slice`, `toSorted`, `toReversed`, `toSpliced` o `with`.
- Usa `reduce` solo cuando realmente necesites acumular; para transformar o filtrar, `map` y `filter` suelen ser mas legibles.
- Recuerda que `find` devuelve `undefined` si no encuentra nada; valida ese caso antes de acceder a propiedades.

Mini reto

Dado el array de productos, filtra los productos activos, calcula el total de sus precios y ordenalos de menor a mayor sin modificar el array original.

```
const productos = [  
  { nombre: "Laptop", precio: 1200, activo: true },  
  { nombre: "Mouse", precio: 25, activo: true },  
  { nombre: "Teclado", precio: 80, activo: false }  
];
```

```
const activosOrdenados = productos
  .filter(p => p.activo)
  .toSorted((a, b) => a.precio - b.precio);

const total = activosOrdenados.reduce((suma, p) => suma + p.precio, 0);
```